

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1715

NAME: P.L.OAVIKSHA

REG:NO: 192572001

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)

Assessment Tool Weightage: 20%

QUESTIONS: 5

Duration: 1 Hour

Total Marks: 50

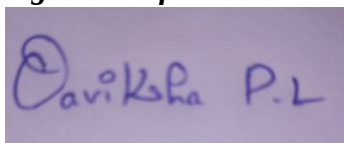
Annexure A

Scenario Based Assignment

Code of Conduct:

*I P.L.OAVIKSHA(192572001) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:



Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

- i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty
- ii. Explain how A* would handle the same situation, including how it balances cost and heuristic
- iii. Analyze the impact of heuristic accuracy on both algorithms
- iv. Discuss how dynamic changes (blocked paths) affect search performance
- v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

2. Scenario: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

- i. Explain how Hill Climbing would work in this scenario and identify its limitations
- ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation
- iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations
- iv. Recommend the best approach and justify your choice based on scalability and adaptability

3. Scenario: An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

- i. Explain why this problem requires an online search agent instead of offline search
- ii. Analyze the challenges of partial observability and dynamic environments
- iii. Describe how the rover builds and updates its internal model
- iv.

Compare online search with uninformed and informed search strategies

- v. Justify the most suitable approach considering uncertainty, adaptability, and safety

4. Scenario: A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

- i. Clearly define variables, domains, and constraints with explanation
- ii. Explain how backtracking search works in this scenario
- iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency
- iv. Analyze real-world challenges and justify the best strategy for scalability

5. Scenario: Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must compete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

- i. Explain how the Minimax algorithm models this problem
- ii. Analyze the computational challenges of large game trees
- iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning
- iv. Design an evaluation function and justify the factors considered
- v. Recommend a strategy for balancing optimality and real-time performance

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

CO1 – ASSESSMENT TOOL 3

Descriptive Written Test

COURSE NAME: Artificial Intelligence

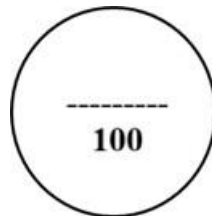
COURSE CODE: CSA17

QUESTIONS: 2

Duration: 45 Minutes

Total Marks: 20

Criteria	Weightage	Q1 (10)	Q2 (10)	Total Marks (20)
Understanding of Concepts	25% (2.5)	/2.5	/2.5	/5
Explanation	25% (2.5)	/2.5	/2.5	/5
Application	20% (2)	/2	/2	/4
Organization	15% (1.5)	/1.5	/1.5	/3
Accuracy	10% (1)	/1	/1	/2
Clarity	5% (0.5)	/0.5	/0.5	/1
Total per Question	10 Marks	/10	/10	/20



Q1. A Smart Home Automation System Using Intelligent Agents

INTRODUCTION

Artificial Intelligence (AI) enables machines to make intelligent decisions by sensing their environment and performing suitable actions. One of the best real-life applications of AI is a Smart Home Automation System, where intelligent agents automatically manage lighting, temperature, security, and home appliances based on sensor inputs and user preferences.

The smart home continuously receives data from different sensors such as motion sensors, temperature sensors, smoke detectors, door sensors, cameras, and smart meters. Based on this information, the AI agent decides whether to switch ON/OFF appliances, adjust room temperature, activate alarms, or notify the homeowner. Over time, the system learns user habits and improves its decision-making ability.

Components of Smart Home System

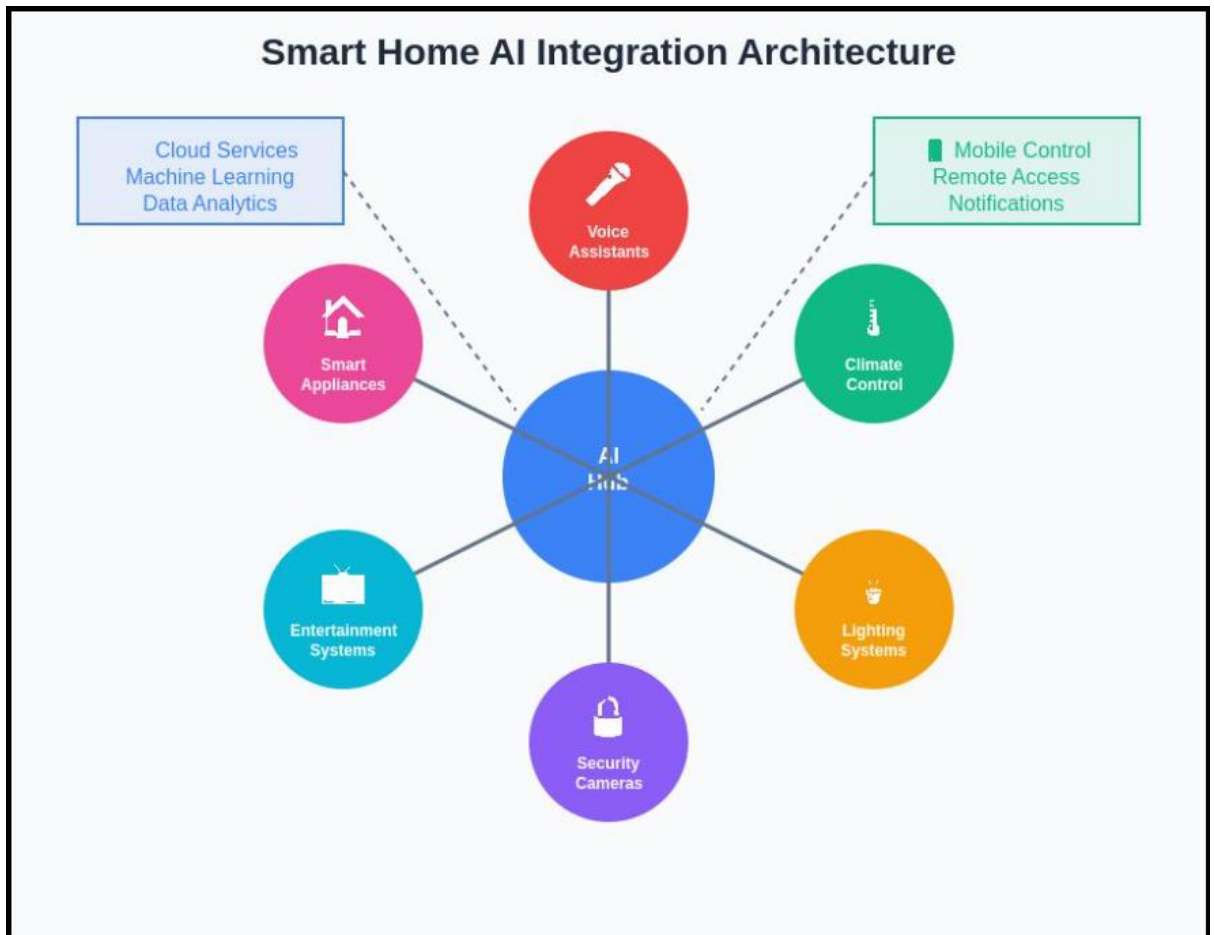
The major components are:

- Motion Sensor
- Temperature Sensor
- Smoke Sensor
- Door Lock Sensor
- Smart Camera
- Smart Lights
- Smart Fan and Air Conditioner
- Wi-Fi Network
- Cloud Database
- Mobile Application
- Intelligent Agent

Working of Smart Home

- Sensors collect environmental information.
- Information is sent to the AI Agent.
- The agent analyzes the current situation.

- It compares the data with predefined rules or learned preferences.
- The best action is selected.
- Appliances perform the required action.
- Feedback is stored for future learning.



Types of Intelligent Agents

1. Simple Reflex Agent

Definition

A Simple Reflex Agent works only on the current perception. It follows predefined IF–THEN rules without considering previous experiences.

Working

Example rules:

IF Motion Detected → Light ON

IF No Motion → Light OFF

IF Temperature > 30°C → AC ON

IF Door Open at Night → Alarm ON

The agent immediately responds whenever these conditions are satisfied.

Advantages

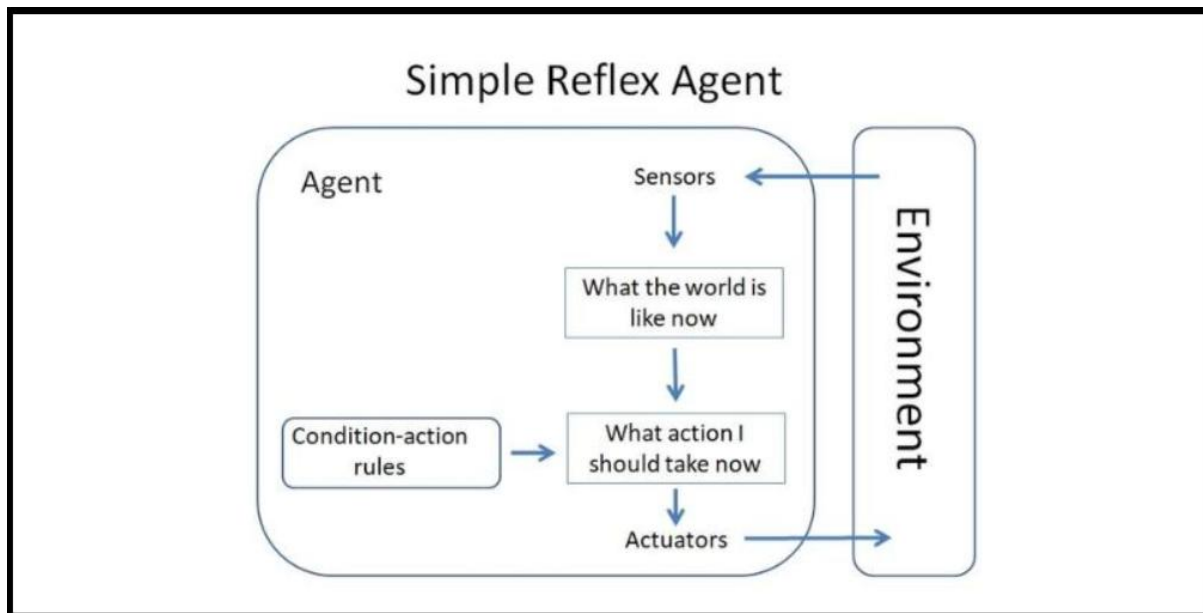
- Very fast response
- Easy to implement
- Low computational cost
- Suitable for simple automation

Disadvantages

- Cannot remember previous events
- Cannot learn user behavior
- Performs poorly in complex environments

Applications

- Automatic street lights
- Motion sensor lights
- Automatic doors
- Fire alarms



2. Model-Based Reflex Agent

Definition

A Model-Based Agent maintains an internal model of the environment. It remembers previous states and uses them while making decisions.

Working

Suppose a motion sensor temporarily fails.

The agent remembers that someone entered the room a few seconds earlier.

Instead of switching OFF the lights immediately, it keeps them ON until another confirmation is received.

Advantages

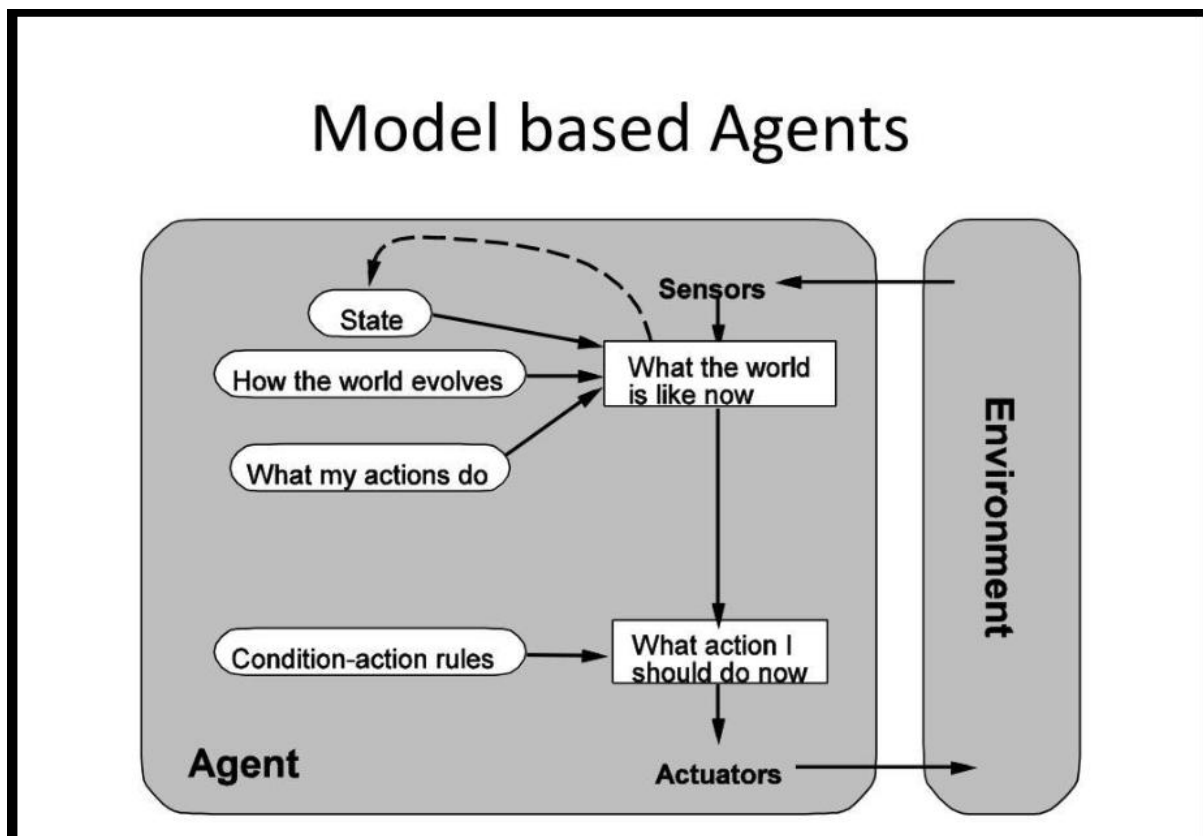
- Handles incomplete information
- More reliable
- Better decision-making
- Reduces unnecessary switching

Disadvantages

- Requires memory
- More complex than simple reflex agents

Applications

- Smart security systems
- Traffic control
- Industrial automation



3. Goal-Based Agent

Definition

A Goal-Based Agent performs actions to achieve a specific objective.

Example Goals

- Maintain room temperature between 22°C–24°C
- Reduce electricity usage
- Maximize user comfort
- Ensure home security

Working

If room temperature becomes 29°C,

the agent considers several possible actions:

- Open smart windows
- Turn ON fan
- Turn ON AC
- Close curtains

Finally, it selects the action that best achieves the desired goal.

Advantages

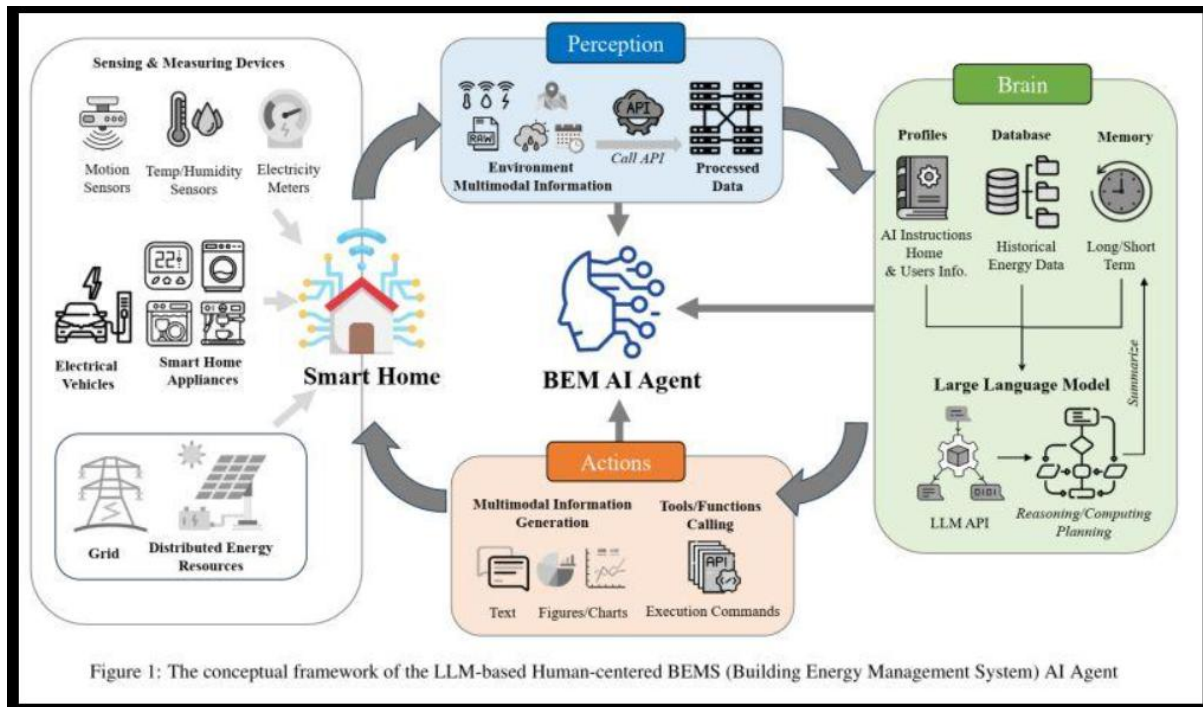
- Intelligent planning
- Flexible
- Suitable for dynamic environments

Disadvantages

- Slower than reflex agents
- Requires search algorithms

Applications

- Smart cities
- Robot navigation
- Autonomous vehicles



4. Utility-Based Agent

Definition

A Utility-Based Agent selects the action that provides the maximum overall benefit instead of only achieving a goal.

Utility Factors

- User comfort
- Electricity cost
- Security
- Energy consumption
- Weather conditions

Example

Suppose

Temperature = 27°C

Electricity Price = High

Instead of running the AC at maximum speed,

the system may:

- Reduce AC speed
- Switch ON ceiling fan
- Close curtains

This provides the highest utility by balancing comfort and cost.

Advantages

- Produces optimal decisions
- Balances multiple objectives
- Learns user preferences
- Very efficient

Disadvantages

- Complex calculations
- High processing requirements

Applications

- Smart energy management
- Healthcare
- Self-driving cars
- Smart buildings

Comparison of Intelligent Agents

Feature	Simple Reflex	Model-Based	Goal-Based	Utility-Based
Memory	No	Yes	Yes	Yes
Learning	No	Limited	Moderate	High
Planning	No	Limited	Yes	Yes
Decision Quality	Low	Medium	High	Very High
Complexity	Low	Medium	High	Very High
Suitable for Smart Home	Basic	Good	Better	Best

Hybrid Intelligent Agent

A Hybrid Agent combines multiple intelligent agent types.

Benefits

- Learns user habits
- Reduces electricity consumption
- Improves comfort
- Provides better security
- Makes intelligent decisions
- Adapts to changing environments

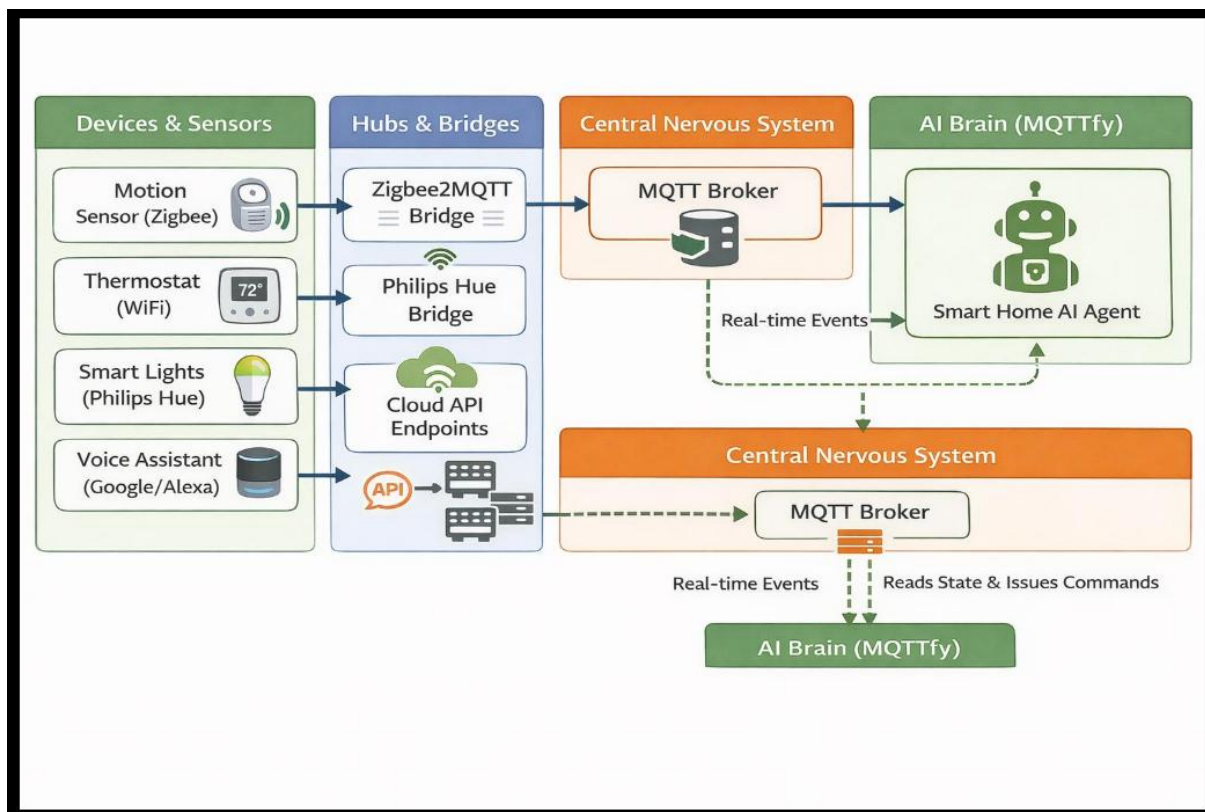
Advantages of Smart Home AI

- Saves electricity
- Improves security
- Provides convenience
- Reduces human effort

- Supports remote monitoring
- Learns user preferences
- Faster response
- Better energy management

Disadvantages

- High installation cost
- Internet dependency
- Privacy concerns
- Cybersecurity risks
- Complex maintenance



Conclusion

Among all intelligent agents, the Hybrid Approach (Goal-Based + Utility-Based Agent) is the most suitable for smart home automation because it combines planning with optimization. It continuously learns user preferences, reduces energy consumption, enhances security, and provides maximum comfort, making the home smarter, safer, and more efficient.

Q2. A Robot is Placed in an Unknown Maze – Uninformed Search Strategies (UCS & IDS)

Introduction

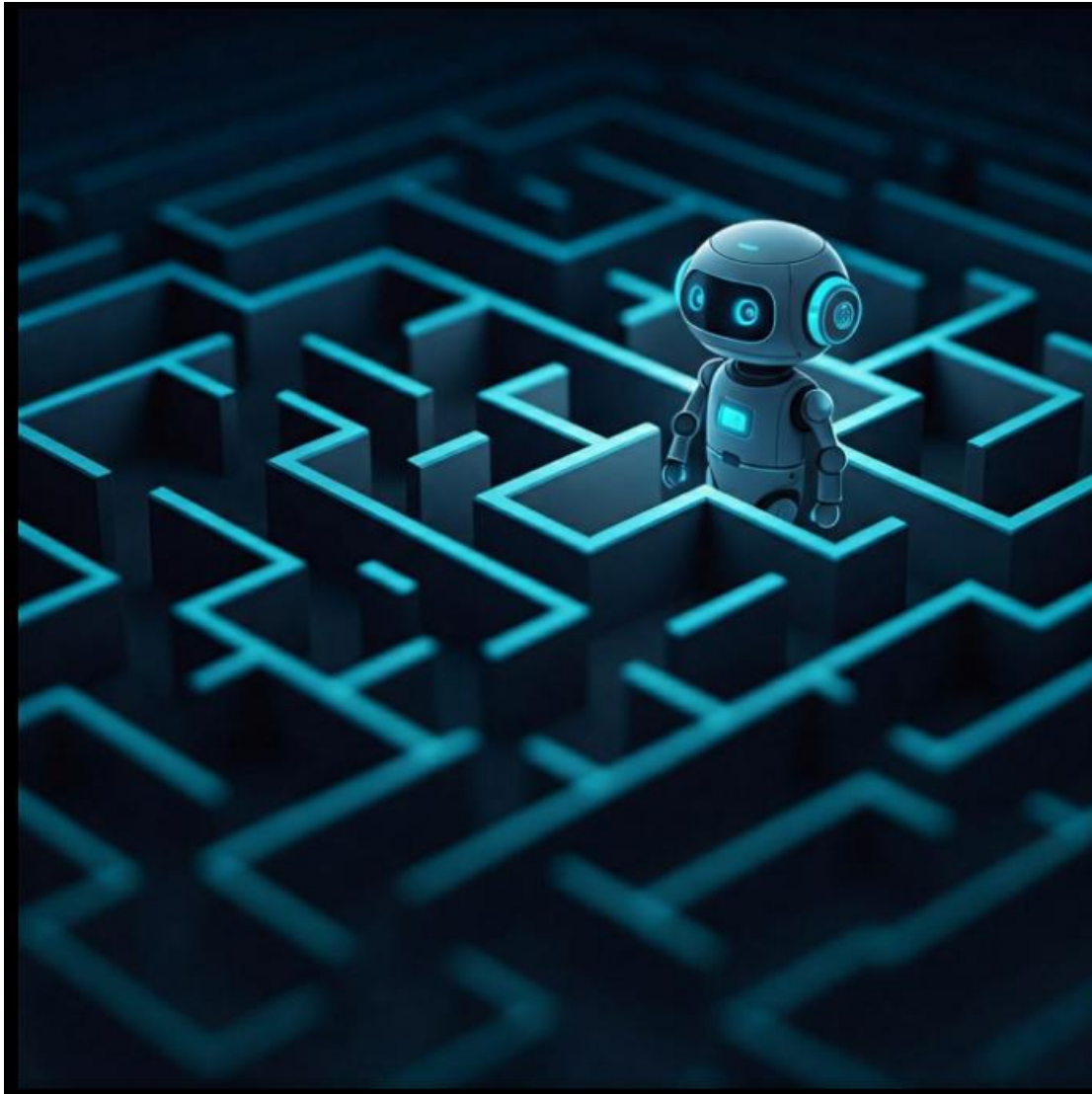
Artificial Intelligence (AI) enables robots to solve problems by searching for solutions intelligently. One important problem is maze navigation, where a robot is placed inside an unknown maze and must find the exit without any prior knowledge of the environment.

Since the robot does not know the location of the exit or the structure of the maze, it must explore different paths systematically. This type of problem is solved using Uninformed Search Algorithms, also known as Blind Search Algorithms, because they do not use heuristic (estimated) information.

Two important uninformed search algorithms are:

- Uniform Cost Search (UCS)
- Iterative Deepening Search (IDS)

These algorithms help the robot find a path while making decisions based only on the available information.



Robot Maze Problem

Problem Description

A robot starts at the entrance of a maze.

Its objective is:

- Find the exit
- Avoid unnecessary movement
- Minimize travel cost
- Reach the goal safely

Components of the Problem

Initial State

Robot starts at the entrance.

Goal State

Robot reaches the exit.

Actions

- Move Up
- Move Down
- Move Left
- Move Right

Path Cost

Each movement has an associated cost such as:

- Distance
- Time
- Battery usage

Uninformed Search

Definition

Uninformed Search is a search technique where the agent has no knowledge about which path is better. It explores the search space using only the problem definition.

Characteristics

- No heuristic function
- Explores nodes systematically
- Guaranteed to find a solution if one exists (depending on the algorithm)
- Suitable for unknown environments

Uniform Cost Search (UCS)

Definition

Uniform Cost Search is an algorithm that always expands the node with the lowest cumulative path cost.

Instead of selecting the nearest node, UCS selects the path that has the minimum total cost from the start.

Working Principle

1. Start from the initial node.
2. Insert it into a priority queue.
3. Expand the node having the smallest path cost.
4. Add its neighboring nodes with updated costs.
5. Continue until the goal is reached.

Example

Suppose the robot has two paths:

Path A

Entrance $\rightarrow$ A $\rightarrow$ Exit

Cost = 18

Path B

Entrance $\rightarrow$ B $\rightarrow$ C $\rightarrow$ Exit

Cost = 12

Although Path B has more steps, UCS selects it because the total cost is lower.

Algorithm

Start



Priority Queue



Select Lowest Cost Node



Expand Node



Update Cost



Goal Found?



Yes → Stop

No → Continue

Advantages of UCS

- Finds the optimal solution.
- Always chooses the least-cost path.
- Works for different path costs.
- Complete algorithm.
- Suitable for navigation systems.

Disadvantages

- High memory usage.
- Slow for very large search spaces.
- Explores many unnecessary nodes.
- Computationally expensive.

Applications

- GPS Navigation
- Robot Path Planning
- Network Routing
- Logistics
- Game AI

Time Complexity

$$O(b^{(1+C/\epsilon)})^*$$

where

b = Branching factor

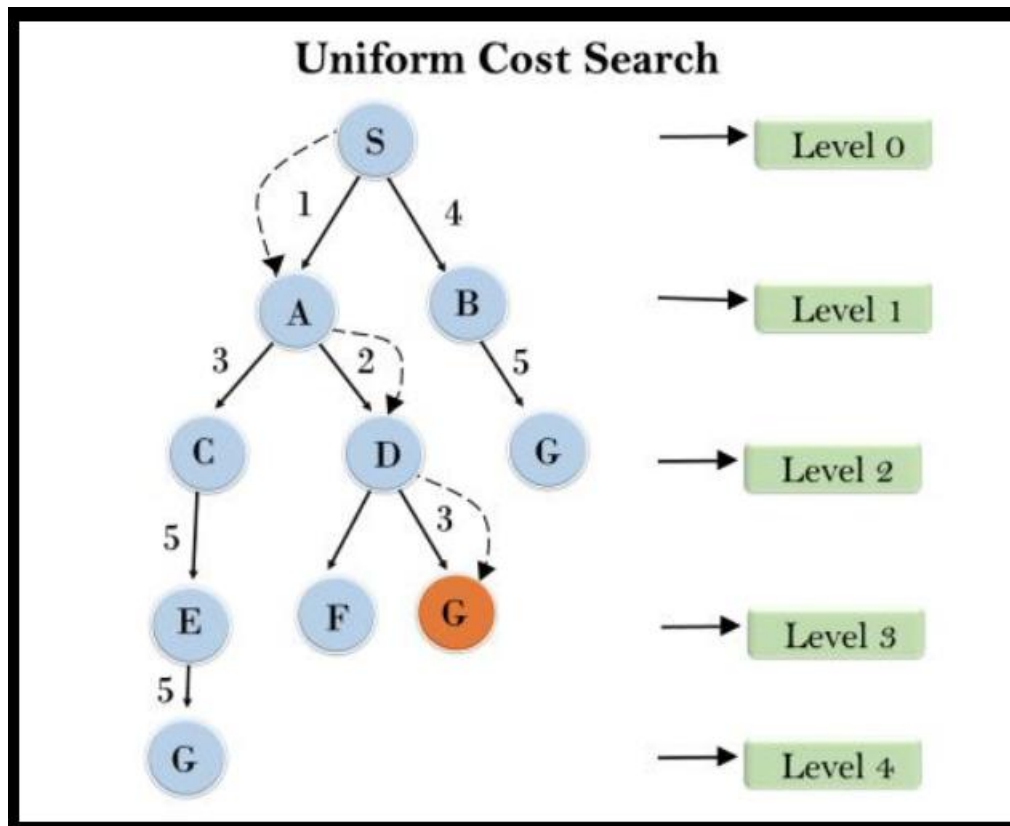
C* = Cost of optimal solution

ϵ = Minimum step cost

Space Complexity

$$O(b^{(1+C/\epsilon)})^*$$

Memory requirement is very high because UCS stores every generated node.

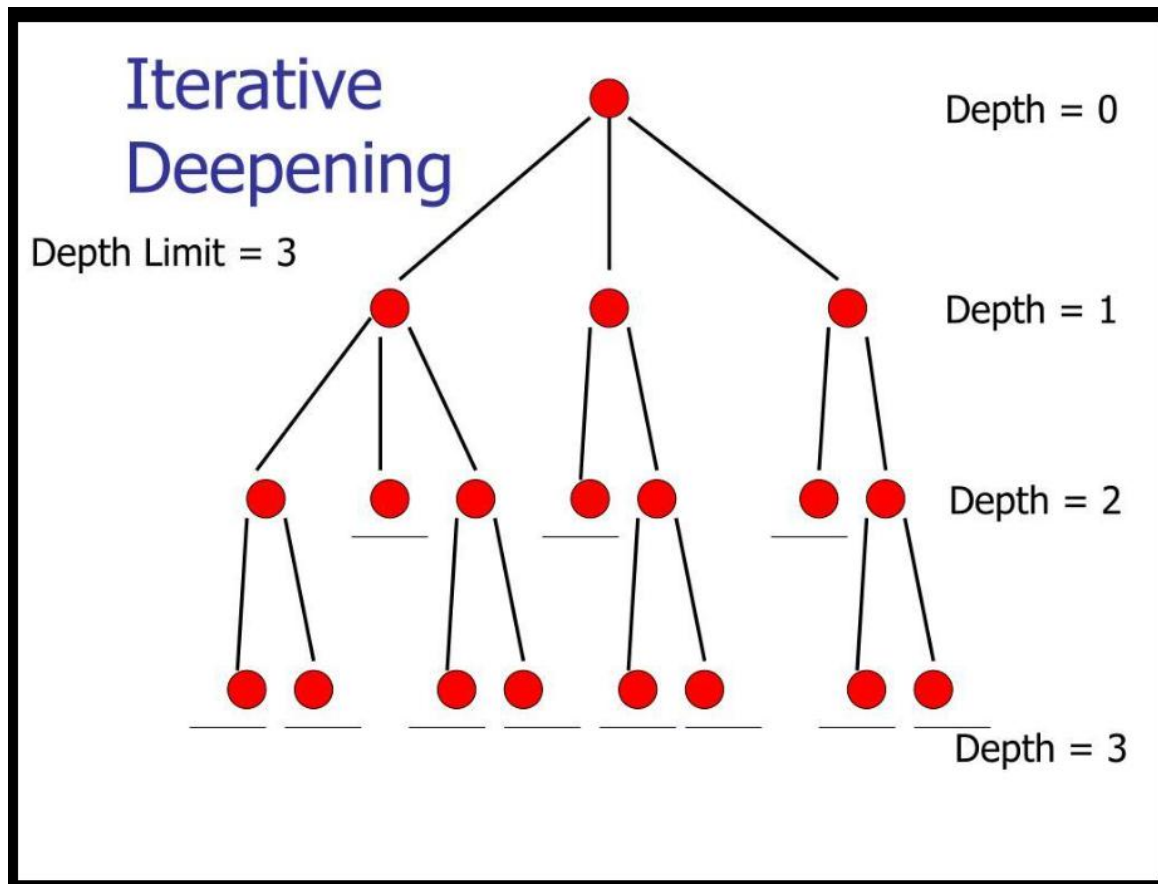


Iterative Deepening Search (IDS)

Definition

Iterative Deepening Search combines the advantages of Depth First Search (DFS) and Breadth First Search (BFS).

It performs repeated depth-limited searches, increasing the depth limit step by step until the goal is found.



Working Principle

Search Depth = 0

If goal not found

↓

Search Depth = 1

↓

Search Depth = 2

↓

Search Depth = 3

↓

Continue until exit is found.

Example

Suppose the exit is at depth 4.

IDS searches:

Depth 0



Depth 1



Depth 2



Depth 3



Depth 4 Robot finds the exit.

Algorithm

Depth = 0



Perform Depth Limited Search



Goal Found?



No



Depth++



Repeat

Advantages

- Requires very little memory.
- Complete algorithm.
- Suitable when goal depth is unknown.
- Easy to implement.
- Better than DFS in unknown environments.

Disadvantages

- Repeats previous searches.
- Slightly slower due to repeated exploration.
- Not optimal when path costs differ.

Applications

- Robot Navigation
- Puzzle Solving
- Web Crawlers
- Game Search
- AI Planning

Time Complexity

$$O(b^d)$$

where

b = Branching factor

d = Depth of solution

Space Complexity

$O(bd)$

Much smaller than UCS.

Comparison Between UCS and IDS

Feature	Uniform Cost Search (UCS)	Iterative Deepening Search (IDS)
Search Type	Cost-based	Depth-based
Uses Cost	Yes	No
Optimal	Yes	No (if costs vary)
Complete	Yes	Yes
Memory Usage	High	Low
Speed	Slower	Faster for shallow goals
Best For	Minimum-cost path	Unknown depth problems

Intelligent Agents in Maze Navigation

1. Simple Reflex Agent

Working

Acts only on current sensor input.

Example:

Obstacle Ahead



Turn Right

It does not remember previous paths.

Limitation

Robot may repeatedly enter the same dead end.

2. Model-Based Agent

Working

Maintains an internal map of explored paths.

It remembers:

- Visited locations
- Obstacles
- Dead ends

Benefits

- Avoids repeated exploration.
- Improves navigation efficiency.

3. Goal-Based Agent

Goal

Reach the maze exit.

The robot evaluates different paths and selects actions that move it closer to the goal.

Advantages

- Intelligent planning
- Efficient decision-making
- Flexible

4. Utility-Based Agent

Working

Considers multiple factors simultaneously:

- Shortest distance
- Battery level
- Time taken
- Safety
- Energy consumption

The robot chooses the path with the highest overall utility.

Example

If one path is shorter but dangerous, while another is slightly longer but safer, the utility-based agent selects the safer route.

Best Agent and Search Strategy

Recommended Combination

Goal-Based Agent + Uniform Cost Search (UCS)

Justification

- The Goal-Based Agent keeps the robot focused on reaching the exit.
- UCS guarantees the least-cost path.
- Together, they ensure efficient and intelligent navigation.
- Suitable for real-world robots where battery life, travel time, and safety are important.

\

Real-Life Applications

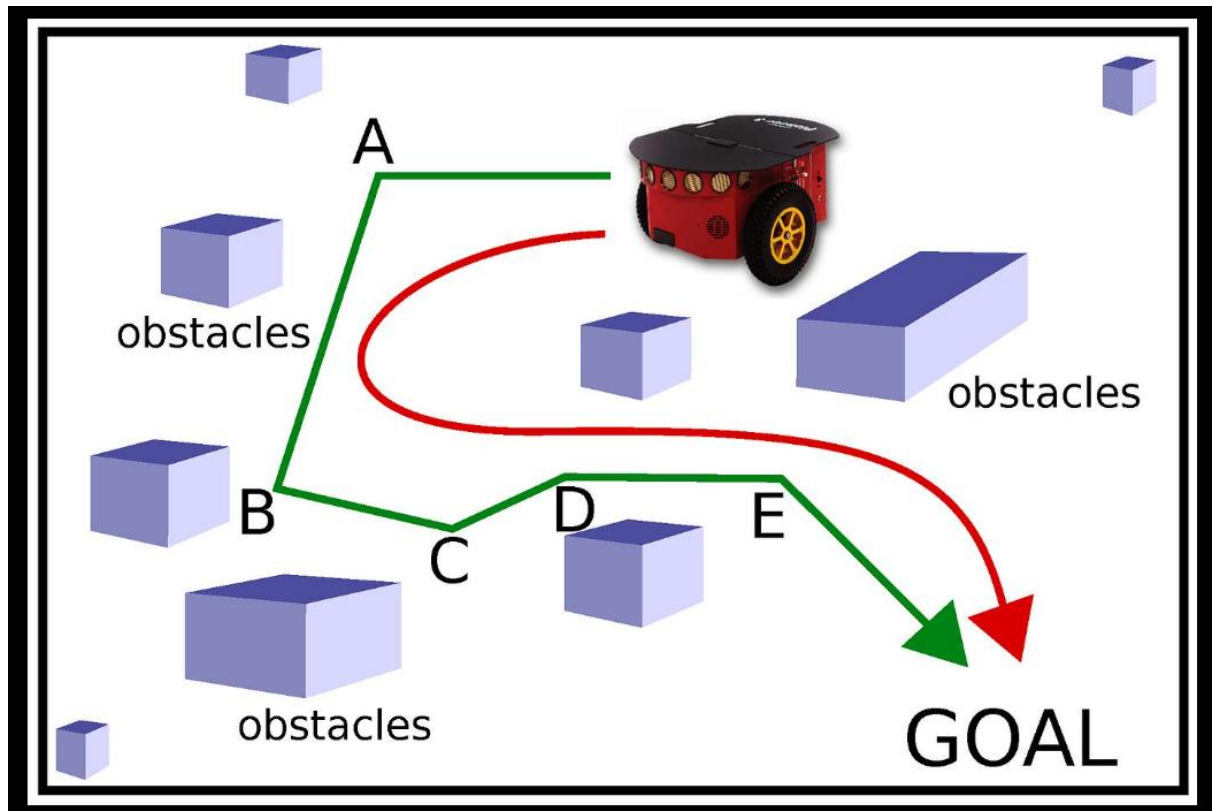
- Autonomous warehouse robots
- Self-driving cars
- Drone navigation
- Rescue robots
- Space exploration rovers
- Delivery robots
- Military surveillance robots
- Hospital service robots

Advantages of Using AI Search in Robots

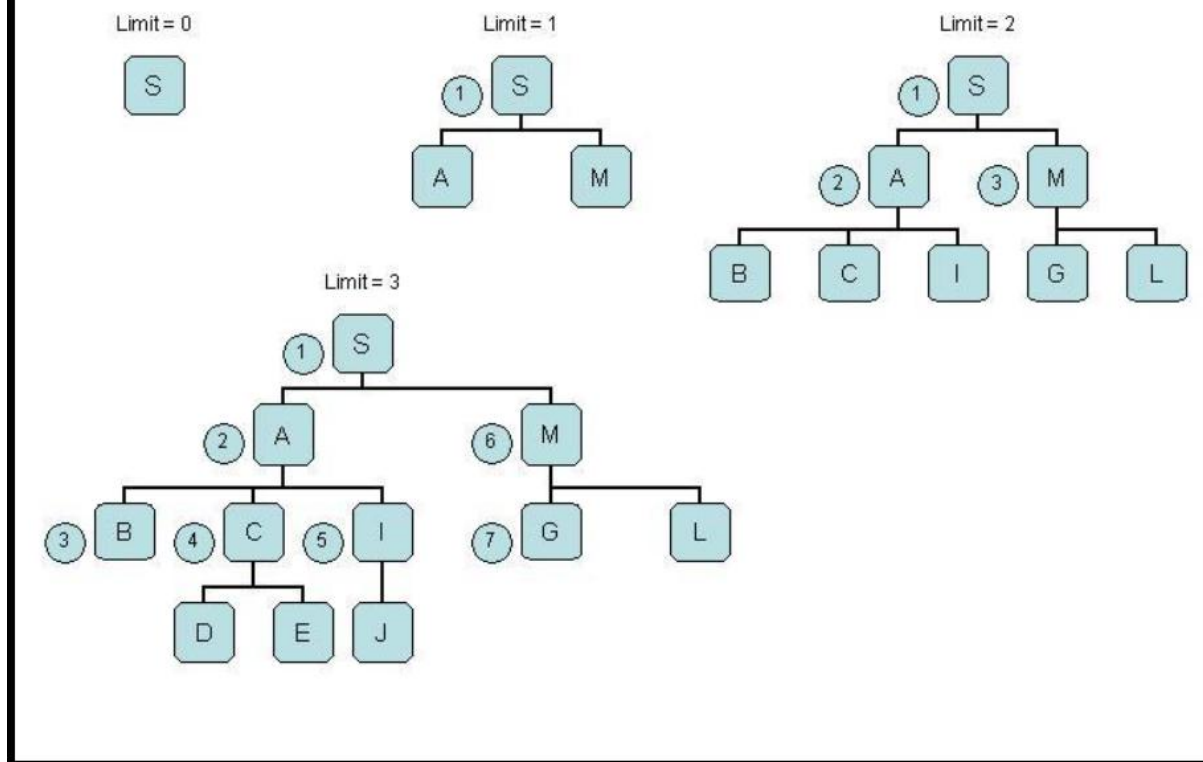
- Finds efficient paths.
- Reduces travel time.
- Saves battery power.
- Avoids unnecessary movements.
- Improves decision-making.
- Works in unknown environments.
- Increases navigation accuracy.

Limitations

- Large search spaces require more computation.
- UCS consumes significant memory.
- IDS repeats searches, increasing execution time.
- Performance decreases in highly complex mazes.



Iterative Deepening Search



Conclusion

Uninformed search algorithms are essential for solving navigation problems when no prior knowledge of the environment is available. Uniform Cost Search (UCS) guarantees the least-cost path, making it ideal for applications where travel cost, energy, or distance matters. Iterative Deepening Search (IDS) is memory-efficient and suitable when the depth of the solution is unknown. For robot maze navigation, a Goal-Based Intelligent Agent combined with Uniform Cost Search provides the best balance of intelligent planning, optimal path selection, and reliable performance in real-world scenarios.